Capítulo 5: Git en Entornos Distribuidos

Actividad 2 - Basado en Pro Git (Chacon & Straub, 2014) - Juan Manuel Gomez Perez - 43972

Introducción

Este capítulo describe flujos de trabajo colaborativos, cómo contribuir a un proyecto ajeno y cómo mantener un proyecto que recibe contribuciones.

5.1 Flujos de trabajo distribuidos

- Flujo centralizado: un único repositorio central al que todos envían cambios (push), haciendo antes pull para integrar lo de los demás.
- Flujo del gestor de integración: cada desarrollador tiene un repositorio público propio; un gestor incorpora los cambios al repositorio oficial. Es el modelo de los forks y pull requests.
- Flujo de dictador y tenientes: variante para proyectos muy grandes, con jerarquía de responsables de distintas áreas (como el kernel de Linux).

5.2 Contribuir a un proyecto

Guías para las confirmaciones

- Evitar espacios en blanco innecesarios (git diff --check).
- Hacer confirmaciones pequeñas, con un cambio lógico cada una.
- Escribir buenos mensajes: una línea de resumen breve (unos 50 caracteres), una línea en blanco y una explicación del porqué del cambio.

Contribución mediante fork

```
# 1. Hacer fork en la web y clonar el fork
git clone <url-del-fork>
git switch -c arreglo-bug
# 2. Trabajar, confirmar y publicar la rama
git push origin arreglo-bug
# 3. Abrir un pull request desde la web
```

Contribución por correo

```
git format-patch -M origin/main   # genera archivos .patch
git send-email *.patch
```

También existe git request-pull, que genera un resumen para pedir al mantenedor que traiga los cambios desde un repositorio público.

5.3 Mantener un proyecto

- Trabajar con ramas temáticas para aislar cada contribución recibida.

- Aplicar parches recibidos: git apply (solo aplica los cambios) o git am (aplica y crea la confirmación conservando el autor).
- Integrar cambios remotos: git remote add, git fetch y git merge.
- Elegir confirmaciones concretas con git cherry-pick.
- Usar rebase para mantener el historial lineal.

```
git apply cambio.patch
git am cambio.patch
git cherry-pick <hash>
```

Publicar versiones

```
git tag -s v1.5 -m "Versión 1.5"      # etiqueta firmada
git describe main                     # nombre legible de la versión
git shortlog --no-merges main         # resumen de autores
git archive main --prefix=proyecto/ -o proyecto.zip
```

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2